CLAUDE CODE — COMPLETE CHEAT SHEET

Architecture · Extensions · Commands · Shortcuts · Config — March 2026 Edition

7-Layer Extension Stack

Your customization pyramid

- **CLAUDE.md** — Project memory
- **Rules** — Conditional rules
- **Skills** — On-demand expertise
- **Hooks** — Deterministic control
- **Subagents** — Isolated workers
- **Agent Teams** — Parallel coordination
- **MCP Servers** — External integrations

Layers 1–4 deterministic · Layers 5–7 probabilistic

CLAUDE.md — Project memory

- Always loaded at session start
- Survives `/compact` — re-injected each time
- Keep under **200 lines**; progressive disclosure
- Run `/init` to auto-generate from codebase
- Token limit: ~500 lines max effective

3 SCOPE LEVELS

`~/.claude/CLAUDE.md`	personal (all projects)
`.claude/CLAUDE.md`	project (git-tracked)
`CLAUDE.md`	project root (alt)

Skills — Auto-loaded expertise

- File: `.claude/skills/NAME/SKILL.md`
- Auto-invoked by Claude via description
- User-invoked as `/skill-name`
- Supports `$ARGUMENTS` placeholder
- `context: fork` runs in subagent
- Follows Agent Skills Open Standard

```
---
name: deploy
description: Deploy to prod
allowed-tools: "Bash(npm:*)"
---
```

Hooks — DETERMINISTIC ntrol — 100% reliable

EVENT	TRIGGER / POWER
`PreToolUse`	Before tool → Block / modify
`PostToolUse`	After tool → Lint / validate
`Notification`	Agent notifies → Alert / log
`Stop`	Agent completes → Cleanup

HANDLER TYPES

command · http · prompt · agent

EXIT CODE SEMANTICS

- Exit 0 → Allow / proceed
- Exit 1 → Warning (non-blocking)
- Exit 2 → Block execution

EXAMPLE HOOK

```
"PostToolUse": [{
  "matcher": "Write",
  "hooks": [{"type":"command",
    "command":"prettier $FILE"}]
}]
```

Subagents — Isolated workers

- Independent context window
- Own tools and execution env
- **No recursion** — can't nest
- Report results to parent
- Built-in: `Explore`, `Plan`
- Custom: `.claude/agents/NAME.yml`

```
---
name: code-reviewer
model: opus
allowed-tools: Read,Grep,Glob
---
```

Agent Teams — Feb 2026

Parallel coordination

- Multiple Claude sessions coordinated
- One lead + N teammates
- Direct messaging between them
- Git worktrees for file isolation
- Configured via prompt (not YAML)

```
claude --worktree feat --tmux
claude --teammate-mode in-process
```

Plugins — Distributable bundles

- Bundle: skills + hooks + agents + MCP
- Namespaced: `/my-plugin:hello`
- Install via `/plugins` or `npx`
- Share across teams via GitHub

```
/plugin marketplace add owner/repo
npx skills add github.com/repo
```

POPULAR PLUGINS

Superpowers · frontend-design · Remotion

MCP Servers — External integrations

- Config: `.mcp.json` (project/global)
- Transports: `stdio`, `sse`, `http`
- Elicitation API for user input
- Manage with `/mcp` in session

```
claude mcp add github \
  npx @mcp/server-github
claude mcp add --transport sse \
  name https://url.com/sse
```

Decision Guide — When to Use What — 10-second decision tree

CLAUDE.md	→ Always-on memory & rules	Subagents	→ Isolated execution
Rules	→ Conditional / path-specific	Agent Teams	→ Parallel coordination
Skills	→ Repeatable workflows	MCP	→ External integrations
Hooks	→ Deterministic safety rails	Plugins	→ Package & share

CLAUDE.md & Hooks = deterministic (always run) · Skills & Agents = probabilistic (Claude uses judgment)

File System

Project & personal config

```
.claude/
  ■■ CLAUDE.md
  ■■ settings.json
  ■■ settings.local.json
  ■■ rules/
  ■■ skills/
  ■■ agents/
  ■■ auto/
.mcp.json
```

Hook Patterns — Worth stealing

- Auto-format
- Pre-commit
- Slack alert
- No force-push
- Block rm -rf
- Security
- Auto-lint
- Type-check

Context Mgmt — Stay fast, stay focused

- `/compact` at 80% (auto at 95%)
- `/clear` when switching tasks
- `@file.ts` for targeted context
- Check: `/cost` or `!tokens`

MODEL STRATEGY

Opus → complex · Sonnet → 80% tasks · Haiku → quick

Bundled Skills — Out of the box

`/btw`	Side question mid-response
`/claude-api`	Anthropic API help
`/debug`	Debug current issue
`/loop`	Iterative refinement
`/simplify`	3-agent code review

by AI Agents Accelerator

CLAUDE CODE — Commands, Shortcuts & Tips

Essential Keyboard Shortcuts

Memorize these — they change everything

SESSION CONTROL

- `Shift` + `Tab` — Normal → Auto-accept → Plan
- `Ctrl` + `F` — Kill all background agents
- `Ctrl` + `C` — Cancel current response
- `Ctrl` + `G` — Open prompt in $EDITOR
- `Opt` + `P` — Quick model picker
- `Opt` + `T` — Toggle extended thinking

CONTEXT

- `@` — file.ts — add file to context

EDITING & NAVIGATION

- `\` + `Enter` — Newline in prompt
- `Ctrl` + `A` — Start of line
- `Ctrl` + `E` — End of line
- `Ctrl` + `U` — Clear line
- `Ctrl` + `W` — Delete word backward

MODES

- `/vim` — Full vim keybindings in prompt
- `/btw` — Side question (no context impact)
- `/fast` — Speed mode (same model)

CLI Flags

Terminal launch options

`-p "prompt"`	Non-interactive (run & exit)
`--model opus`	Set model at launch
`--worktree`	Isolated git worktree
`--tmux`	Launch in tmux session
`--debug`	Show hook execution
`--resume`	Continue last session
`--dangerously-skip-permissions`	Auto-accept all

Slash Commands

The 15 you'll use daily

`/compact`	Summarize context (+ focus hint)
`/clear`	Wipe history, start fresh
`/init`	Generate CLAUDE.md from codebase
`/model`	Switch opus / sonnet / haiku
`/cost`	Token usage & spend
`/fork`	Clone conversation to experiment
`/undo`	Roll back to previous state
`/btw`	Side question (no context impact)
`/plan`	Think before acting
`/fast`	Speed-optimized mode
`/simplify`	3-agent parallel code review
`/permissions`	View / edit permissions
`/mcp`	Manage MCP connections
`/hooks`	Configure hooks interactively
`/doctor`	Run diagnostics

Prompt Formula & Workflow

Structure every request like this

PROMPT FORMULA (WHAT/WHERE/HOW/VERIFY)

- **WHAT**: Concrete deliverable
- **WHERE**: File paths to touch
- **HOW**: Constraints, approach, patterns
- **VERIFY**: Success criteria or test to run
- Specify what **NOT** to do

DAILY WORKFLOW (9 STEPS)

- `claude` > `/status` > `Shift+Tab` (Plan)
- Describe task > Review diff > Accept/Reject
- Verify (run tests) > Commit > `/compact`
- `/clear` between tasks, `/compact` at 70%
- Use `ultrathink` for hard problems (Opus)

Plans & Models

Pricing as of March 2026

PLANS

Pro	$20/mo	Limited usage
Max	$100/mo	Unlimited usage
Team	$30/user/mo	Collaboration features
Enterprise	Custom	SSO, audit, compliance

MODELS (MARCH 2026)

Opus 4.6	Best reasoning, complex tasks
Sonnet 4.6	Fast, cost-efficient, 80% tasks
Haiku 4.5	Quickest, simple queries

Default effort: medium for Opus (since Mar 2026)

Quick Install & First Steps

```
curl -fsSL https://claude.ai/install.sh | bash  # Install (Mac/Linux)
irm https://claude.ai/install.ps1 | iex          # Install (Windows)
cd my-project && claude                           # Launch in project
/init                                             # Generate CLAUDE.md
Shift+Tab                                         # Enter plan mode
```

Resources & Links

Official Docs	code.claude.com/docs
Skills Hub	skills.sh
Awesome Claude Code	github.com/hesreallyhim/awesome-claude-code
Superpowers Plugin	github.com/obralab/superpowers

by AI Agents Accelerator

CLAUDE CODE — Strategies, Templates & Advanced

Model Switching Strategy

Switch dynamically, save money

WHEN TO SWITCH

- **Sonnet** (default): 80% of tasks, fast, cheap
- **Opus**: Complex architecture, subtle bugs, deep reaso
- **Haiku**: Quick questions, simple edits, trivia
- Switch back to Sonnet when hard part is done

WORKFLOW EXAMPLE

```
# Start with Sonnet (default)
claude
> "Implement OAuth2 flow with PKCE"
/model opus   # Complex task = Opus
# ... feature done ...
/model sonnet # Back to Sonnet
```

Task Decision Tree

What to do for each task type

Simple task
-> Just ask Claude

Complex task
-> Use Plan Mode first

Risky change
-> Shift+Tab x2 (Plan Mode)

Repeating task
-> Create a Skill or Agent

Context full
-> /compact or /clear

Need docs
-> Use Context7 MCP

Deep analysis
-> Use Opus + ultrathink

Remote Control

Control Claude from your phone

- `/rc` or `/remote-control` in session
- Scan QR code with phone
- Continue session from mobile
- Auto-enable in `/config`

TASKLIST (PERSISTENT TASKS)

```
export CLAUDE_CODE_TASK_LIST_ID=\
  "my-project"
claude
> "Create tasks for auth system"
# Resume later in new session
```

Agent Template

.claude/agents/NAME.yml

```
---
name: my-agent
description: Use when [trigger]
model: sonnet
allowed-tools:
  - Read
  - Grep
  - Glob
  - Bash(git:*)
---
# Agent instructions here
# Claude follows these when
# delegating to this agent
```

Skill Template

.claude/skills/NAME/SKILL.md

```
---
name: my-skill
description: Do X when user
  asks for Y or mentions Z.
allowed-tools: "Bash(git:*)"
---
# Skill Instructions
1. First do this
2. Then do that
3. Output in this format

## Examples
- Example usage here
```

Hook Template

settings.json example

```
{ "hooks": {
    "PreToolUse": [{              // Before tool runs
      "matcher": "Bash",          // Which tool to match
      "hooks": [{
        "type": "command",         // command | http | prompt | agent
        "command": "my-script.sh"  // Exit 0=allow, 1=warn, 2=BLOCK
      }]
    }],
    "PostToolUse": [{             // After tool runs
      "matcher": "Write",
      "hooks": [{"type":"command","command":"prettier --write $FILE"}]
    }]
} }
```

Power Tips

- `ultrathink` forces Opus high effort
- `!tokens` debug token breakdown
- `/rc` control from your phone
- `--mcp-debug` debug MCP servers
- `/fork` before risky experiments
- HANDOFF.md for multi-session work
- `/install-github-app` auto PR review

CLAUDE CODE — Complete Reference

All Slash Commands

Complete list - * = new/missing from other guides

`/compact [hint]`	Compress context (add focus)
`/clear`	Wipe history, fresh start
`/init`	Generate CLAUDE.md
`/model`	Switch opus/sonnet/haiku
`/cost`	Token usage & spend
`/status *`	Context %, auth, session info
`/config *`	Open settings panel
`/fork`	Clone conversation
`/undo`	Roll back last change
`/diff *`	Interactive diff viewer
`/btw`	Side question (no context)
`/plan`	Plan mode (think first)
`/fast`	Speed-optimized mode
`/simplify`	3-agent code review
`/review`	(deprecated -> /simplify)
`/debug`	Debug current issue
`/loop`	Iterative refinement
`/batch *`	Parallel multi-worktree exec
`/permissions`	View/edit permissions
`/mcp`	Manage MCP connections
`/hooks`	Configure hooks
`/doctor`	Run diagnostics
`/agents`	Create/manage agents
`/rc *`	Remote control (phone)
`/login`	Re-authenticate
`/install-github-app`	Auto PR reviews
`/keybindings`	Edit keyboard shortcuts
`/rename`	Rename conversation
`/terminal-setup`	Terminal config
`/insights *`	Usage analytics

All Keyboard Shortcuts

Complete list - * = new/missing

`Shift+Tab`	Cycle permission modes
`Ctrl+C`	Cancel response
`Ctrl+F`	Kill background agents
`Ctrl+G`	Open prompt in $EDITOR
`Ctrl+R *`	Search history
`Ctrl+A`	Start of line
`Ctrl+E`	End of line
`Ctrl+U`	Clear entire line
`Ctrl+W`	Delete word backward
`Ctrl+K *`	Delete to end of line

SPECIAL PREFIXES

`Tab *`	Toggle thinking mode
`Esc Esc *`	Rewind menu
`\+Enter`	New line in prompt
`Option+P`	Model picker (Alt+P Win)
`Option+T`	Ext. thinking (Alt+T Win)
`Option+B *`	Move word backward
`Option+F *`	Move word forward
`@file`	Add file to context
`# text *`	Quick add to CLAUDE.md
`! cmd *`	Direct bash execution

CLI Flags & Advanced

Complete flags + pro tips

`-p "text"`	Non-interactive mode
`--model opus`	Set model at launch
`--resume`	Continue last session
`--continue`	Continue specific session
`--worktree`	Isolated git worktree
`--tmux`	Launch in tmux
`--debug`	Verbose logging
`--mcp-debug *`	Debug MCP servers
`--add-dir *`	Add extra directories
`--max-turns *`	Limit turns (non-int.)
`--output-format *`	json/text output
`--dangerously-skip-permissions`	Auto-accept all (use with caution)

ADVANCED TIPS

`ultrathink`	Force Opus high effort
`!tokens`	Debug token breakdown
`# quick note`	Add to CLAUDE.md instantly
`! ls -la`	Direct bash from prompt
`/batch`	Parallel multi-worktree
`/diff`	Review all changes visually
`Esc Esc`	Rewind to any point
`/rc`	Control from phone
`HANDOFF.md`	Multi-session continuity
`/config`	All settings in one panel

Complete Daily Workflow

The 9-step loop

```
1. claude  >  2. /status  >  3. Shift+Tab (Plan)  >  4. Describe task  >  5. Review d
iff (/diff)  >  6. Accept/Reject  >  7. Run tests  >  8. Commit  >  9. /compact at 70%
```

Sonnet for routine | /model opus for complex | /model sonnet when done | /clear between tasks | ultrathink for hard problems

Quick Install & Resources

```
curl -fsSL https://claude.ai/install.sh | bash
irm https://claude.ai/install.ps1 | iex
```

Mac/Linux

Windows (+ Git for Windows)

code.claude.com/docs — Official docs

skills.sh — Skills hub

by **AI Agents Accelerator**